



Xác Định Bài Toán Kinh Doanh Cho AI

AICB-P1 · Ngày 2 · Chọn đúng bài toán, đúng mức tự động hóa, đúng gate kỹ thuật

Tên Giảng Viên

VinUniversity · Phase 1 · Tuần 1 · 2026

HÃY SUY NGHĨ...

“Một startup gọi vốn để build AI agent. Model ổn, demo đẹp, team khỏe. 6 tháng sau vẫn không scale. Vấn đề nằm ở model, hay ở chuỗi quyết định ngay từ đầu?”

Giữ câu hỏi này trong đầu khi học bài hôm nay

1. AI Landscape và anti-patterns
2. AI Product Lifecycle và gate criteria
3. AI feature/agent anatomy
4. Framework chọn đúng mức AI
5. Problem Statement cho AI system
6. Stakeholder, risk owner, business value
7. Discovery và feasibility trước khi code
8. Lab: đề xuất use case + go/no-go

- Chọn được bài toán nên dùng **workflow**, **LLM feature**, hay **agent**
- Dùng **lifecycle + gate + eval mindset** để quyết định go/no-go
- Viết **Problem Statement** có metric, boundary, và cách đo kết quả
- Chốt **feasibility** trước khi team commit công sức implementation

AI project hiếm khi thất bại vì “không có model”; nó thường thất bại vì chọn sai bài toán, sai kiến trúc, hoặc không đặt gate đúng lúc.

Problem Statement + Stakeholder/RACI-lite + AI Readiness + Go/No-Go + next experiment

- 1 use case cụ thể, có actor, workflow hiện tại, và bottleneck rõ ràng
- 1 bộ metric để đánh giá trước khi nói đến model ngon nào
- 1 quyết định rõ: **rule/workflow**, **LLM feature**, hay **agent**

AI Landscape và Cơ Hội

Dùng số liệu để ra quyết định, không dùng trend để hợp thức hóa mọi ý tưởng

01

88%

Tổ chức
đã dùng AI
đều đặn
trong ít nhất
một function

1/3

Báo cáo
đã scale
AI ở mức
chương trình

23%

Đã scale một
hệ thống
agentic
AI nào đó

39%

Báo cáo AI
tác động
đến EBIT

Ý nghĩa cho Dev: có nhiều pilot và demo, nhưng scale thật sự vẫn khó vì bài toán, workflow, eval, và vận hành.

Loại công việc	Giá trị cao khi...	Kiểu giải pháp thường hợp	Lý do
Tác vụ lặp lại, biến thể vừa phải	Volume lớn, SLA rõ, sai được một mức hữu hạn	Workflow + LLM feature	Giá trị đến từ time saved và giảm variance
Tác vụ cần tổng hợp tài liệu / search	Dữ liệu phân tán, con người đang mất nhiều phút mỗi case	RAG / search-assisted feature	AI giảm chi phí truy xuất và tổng hợp
Tác vụ cần nhiều bước, nhiều tool	Cần lập kế hoạch, gọi API, xử lý trạng thái	Agent có guardrails	Chỉ nên dùng khi workflow thực sự động và phức tạp
Tác vụ deterministic, quy tắc rõ	Rule rõ, edge cases ít, cần output ổn định	Rule engine / workflow thường	AI dễ thành overkill, khó debug, khó compliance

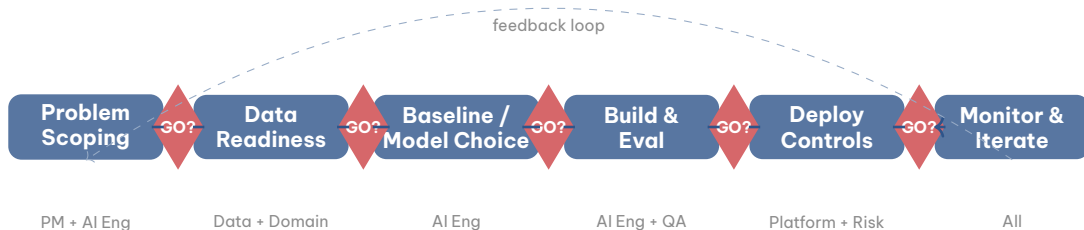
Nguồn: McKinsey 2025 + tổng hợp teaching heuristics cho AI product teams

- ☐ **Trend-first:** Thay đổi theo trend “agent” trước khi rõ actor, workflow, metric
- ☐ **No baseline:** Không có rule/manual baseline để so sánh nhưng vẫn build AI trước
- ☐ **No eval path:** Có demo đẹp nhưng không có bộ test, không biết khi nào đủ tốt để deploy
- ☐ **No owner of failure:** Không rõ ai review output sai, ai rollback, ai chịu compliance
- ☒ **Nguyên tắc:** Dừng AI khi nó tạo giá trị hơn cách đơn giản hơn, không phải vì nó nghe hiện đại hơn

AI Product Lifecycle

Mỗi giai đoạn là một quyết định kỹ thuật và một gate go/no-go

02



Không phải cứ build xong là ship. AI system cần gate trên metric, dữ liệu, chi phí, risk control, và khả năng rollback.

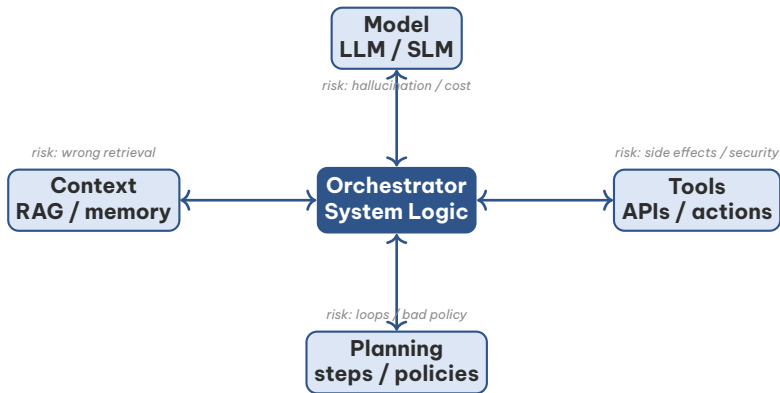
Giai đoạn	Go nếu...	No-Go nếu...
Problem Scoping	Actor, workflow, pain, metric đã rõ	Problem mơ hồ, mô tả solution trước problem
Data Readiness	Có data / logs / docs / SME support để eval	Không có nguồn dữ liệu hoặc dữ liệu quá lệch
Baseline / Model	Heuristic, workflow, hoặc baseline người được xác lập	Chưa biết AI cần tốt hơn cái gì
Build & Eval	Có bộ test, task-level eval, latency/cost budget	Chỉ có demo thử công, không có reproducible eval
Deploy Controls	Có HITL, logging, approval, rollback, owner of failure	Không rõ ai duyệt output, ai dừng hệ thống khi sai

Rules of ML + practical AI product heuristics

- **Không có metric thì không có gate:** chưa đo được thì chưa biết có nên scale hay không.
- **Không có baseline thì không có lợi ích rõ ràng:** AI phải tốt hơn rule, workflow, hoặc human baseline nào đó.
- **Không có eval thì chỉ đang demo:** test infrastructure và test workflow độc lập với model.
- **Không có rollback thì deploy là đánh bạc:** cần owner, logging, và cách dừng hệ thống an toàn.

AI Feature / Agent Anatomy

Mỗi thành phần là một lựa chọn kiến trúc, một failure mode, và một cost driver



4 câu hỏi thiết kế cần hỏi:

- Cần model mạnh tới mức nào?
- Có cần context động hay docs là đủ?
- Có cần planning nhiều bước không?
- Có cần gọi tool / action thật không?

Thành phần	Cần khi...	Failure mode	Control	Cost driver
Model	Task mở, cần suy luận / tạo văn bản	Hallucination, inconsistency	Eval set, guardrails, structured output	tokens / latency
Context	Cần dựa trên tài liệu / trạng thái ngoài	Wrong retrieval, stale context	Retrieval test, freshness, citations	storage / retrieval
Planning	Task nhiều bước, cần ra quyết định trung gian	Looping, over-planning	Step limit, policy, approvals	extra calls
Tools	Cần đọc / ghi hệ thống ngoài	Side effects, prompt injection	sandbox, allowlist, HITL	API calls / failures

PAIR + OpenAI safety + practitioner heuristics

- **Chỉ cần text transform / summarize?** Thử prompt + structured output trước.
- **Cần tra cứu dữ liệu / docs?** Thêm retrieval trước khi nói đến agent.
- **Cần nhiều bước có điều kiện?** Workflow có state có thể đủ.
- **Chỉ khi nào cần tự chọn bước + tự gọi nhiều tool + xử lý feedback loop** mới nghiêm túc xem xét agent.

Lưu ý: Nếu không trả lời được vì sao cần planning và tool autonomy, khả năng cao là bạn chưa cần agent.

Framework Chọn Đúng Bài Toán

Không hỏi “AI làm được không?”; hỏi “mức tự động hóa nào đáng giá ở đây?”

Rule / Script

- Input ổn định
- Logic rõ ràng
- Cần predictability cao
- Compliance nặng

LLM Feature

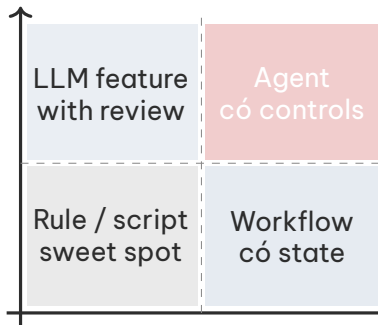
- Input biến thể vừa phải
- Output cần linh hoạt
- Có metric và guardrails
- Human có thể review

Agent

- Nhiều bước, nhiều tool
- Trạng thái thay đổi liên tục
- Cần quyết định động
- Có risk control rõ ràng

Thứ tự ưu tiên thực dụng: bắt đầu từ bên trái, chỉ đi sang bên phải khi giá trị tăng hơn độ phức tạp.

Task ambiguity →



Operational complexity →

5 tín hiệu cần chấm nhanh:

- Determinism cần cao đến đâu?
- Volume lặp lại có đủ lớn không?
- Mức mơ hồ có cần model không?
- Có cần tool / side effect không?
- Nếu sai, có chấp nhận được không?

- ☒ **Value:** Bài toán xảy ra thường xuyên và đang tốn thời gian / tiền / SLA thật sự?
- ☒ **Baseline:** Hiện tại đã có manual hoặc workflow baseline để so sánh?
- ☒ **Eval:** Có metric, sample cases, hoặc logs để đánh giá reproducible?
- ☒ **Tolerance:** Sai số hữu hạn có chấp nhận được, hoặc có HITL ở điểm quan trọng?
- ☒ **Operations:** Có owner, rollback, logging, và policy cho output sai?
- ☐ Dưới 3 câu YES: dừng lại và làm rõ problem / workflow trước khi đầu tư AI

Nên để Rule/Work-flow

Đổi tên file, validate schema, route ticket

- Input khá ổn định
- Rule rõ
- Sai là phí vận hành

Nên là LLM Feature

Trả lời CS có tra cứu tư liệu nội bộ

- Cần tổng hợp nguyên nhân
- Có thể review / cite
- SLA đo được

Chỉ cần agent nếu...

Booking / operations assistant

- Nhiều bước + nhiều tool
- Trạng thái thay đổi
- Có approvals / rollback

Câu hỏi đúng cho team: architecture nào tạo đủ giá trị mà vẫn kiểm soát được risk?

Problem Statement

Từ ý tưởng mơ hồ đến bài toán có metric, boundary, và đường eval

05

Actor / Operator Ai đang làm việc này hằng ngày?

Current Workflow Hiện tại họ xử lý qua những bước nào, dùng tool gì?

Bottleneck Bước nào chậm, lỗi, không nhất quán, hoặc cần tổng hợp quá nhiều?

Impact Tổn thất đo bằng thời gian, chi phí, SLA, error rate, hay conversion nào?

Success Metric Khi nào được coi là thành công? Mức ngưỡng là bao nhiêu?

Operational Boundary Hệ thống được phép làm gì, không được phép làm gì, và điểm nào cần HITL?

Nếu viết xong mà bạn chưa suy ra được test cases, eval metric, và architecture boundary, thì problem statement vẫn chưa đủ chặt.

- ☐ **Solution-first:** “Xây agent để...” trước khi nói rõ workflow hiện tại đang vỡ ở đâu
- ☐ **Không có metric:** Không biết giảm cái gì, tăng cái gì, hay đạt mức nào
- ☐ **Không có boundary:** Không rõ system chỉ để draft, suggest, hay được phép action thật
- ☐ **Scope quá rộng:** “Tự động hóa CS” thay vì một workflow cụ thể có SLA rõ
- ☒ **Mẫu câu đúng:** Actor nào đang gặp bottleneck nào trong workflow nào, gây tổn thất gì, và thành công được đo bằng metric nào?

Problem Statement mẫu

Actor / workflow: Agent CS xử lý ticket tra cứu giao dịch và hồ sơ mở thẻ.

Bottleneck: Mỗi ticket cần tra 4-5 hệ thống và tóm tắt lại cho khách.

Impact: Thời gian xử lý trung bình 8 phút; 40% ticket vượt SLA 5 phút.

Success metric: 80% ticket thuộc top 5 intent được xử lý dưới 2 phút mà không tăng tỉ lệ trả lời sai.

Boundary: Hệ thống chỉ được đề xuất câu trả lời và trích dẫn context; agent CS xác nhận trước khi gửi.

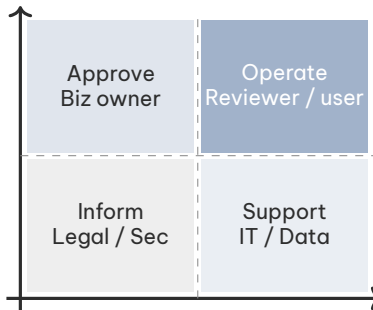
- **Workflow boundary** cho bạn biết cần **feature**, **workflow**, hay **agent**.
- **Success metric** cho bạn biết cần đo SLA, task success, factuality, hay cost/task.
- **Boundary + risk** cho bạn biết cần **HITL**, **approval**, hay chỉ logging là đủ.
- **Top intents / top cases** cho bạn bộ test trước khi mở rộng ra long-tail.

Lưu ý: Problem Statement tốt là cầu nối giữa bài toán kinh doanh và bộ eval kỹ thuật.

Stakeholder và Business Value

Dev cũng phải biết ai dùng, ai review, ai chịu risk, và ai approve deploy

Decision power →



Operational exposure →

4 câu hỏi Dev phải trả lời:

- Ai sử dụng output hằng ngày?
- Ai review output rủi ro cao?
- Ai sở hữu compliance / sự cố?
- Ai có quyền approve hoặc rollback?

Hoạt động	Biz/Ops	AI Eng	Platform/Data	Risk/Legal
Define problem + metric	A/R	C	I	I
Chốt baseline + eval set	C	A/R	R	I
Chọn architecture	C	A/R	C	C
Chốt HITL / approvals	A	C	C	R
Go-live / rollback owner	A	C	R	R

RACI-lite cho giai đoạn đề xuất use case đến pilot

Metric hệ thống cần theo dõi

- Task success rate / groundedness
- P50/P95 latency và cost/task
- Escalation rate / human review load
- Failure mode theo workflow step

Metric business cần tác động

- SLA breach, time-to-resolution
- Cost per case, throughput, retention
- Error rate gây tổn thất
- Tốc độ xử lý top intents

- Không dừng ở mức “agent đạt 87% accuracy”.
- Cần nối tiếp thành: **giảm SLA breach bao nhiêu, giảm cost/ticket bao nhiêu, giảm human handling bao nhiêu.**
- Công thức để dạy trên lớp: **pain today -> workflow metric -> business impact -> architecture choice.**

Discovery và Feasibility

Kiểm tra bài toán, khả năng triển khai, và risk trước khi viết dòng code đầu tiên

- **Pain point là gì?** Xảy ra bao nhiêu lần / ngày / tuần?
- **Workflow hiện tại ra sao?** Tool nào, bước nào, ai hand-off cho ai?
- **Cost của vấn đề là gì?** Mất bao nhiêu phút, tiền, SLA, hay conversion?
- **Nếu AI sai thì sao?** Điểm nào cần HITL, approval, hoặc chỉ để suggest?
- **Ai sẽ nói YES?** Metric nào và risk nào quyết định việc đầu tư?

Lưu ý: Nếu stakeholder không mô tả được workflow hiện tại và failure cost, team đang đề xuất solution trong sương mù.

Technical

- ☐ Có baseline để so sánh?
- ☐ Có data / logs / docs để eval?
- ☐ Latency và cost/task chấp nhận được?
- ☐ Kiến trúc có đơn giản hơn không?

Operational

- ☐ Có logging, monitoring?
- ☐ Có HITL / approvals nếu cần?
- ☐ Có rollback path?
- ☐ Có owner khi output sai?

Business

- ☐ ROI dương ở pilot scope?
- ☐ Risk/compliance rõ ràng?
- ☐ Ops team sẵn sàng đổi workflow?
- ☐ Value đủ lớn để justify độ phức tạp?

Nếu không trả lời được 2-3 câu hỏi ở mỗi cột, bạn chưa sẵn sàng build agent. Có thể bạn mới chỉ sẵn sàng làm workflow nhỏ hoặc data collection pilot.

- **Go:** Problem rõ, baseline rõ, eval rõ, risk controls rõ.
- **Not Yet:** Có pain thật, nhưng thiếu data, thiếu metric, hoặc chưa rõ workflow boundary.
- **No-Go:** Rule-based đã đủ tốt, hậu quả khi sai quá đắt, hoặc change cost lớn hơn value.

Quyết định chuẩn không phải lúc nào cũng là “build”. Rất nhiều dự án AI thành công bắt đầu bằng **không build ngay**, mà bằng đo baseline, gom logs, và thử workflow đơn giản hơn.

Thực Hành

Lab 2: Chọn use case, viết PS, và ra quyết định go/no-go

08

1. Chọn một domain và một workflow cụ thể có pain + metric rõ ràng
2. Chấm nhanh: rule/workflow, LLM feature, hay agent
3. Viết Problem Statement đầy đủ actor, bottleneck, impact, metric, boundary
4. Vẽ stakeholder/RACI-lite và đánh giá feasibility
5. Chốt **Go / No-Go / Not Yet** kèm next experiment nhỏ nhất

Dùng AI để brainstorm use case và draft artifact, nhưng con người phải sửa metric, boundary, và risk controls cho đúng với workflow thật.

Mục tiêu: Chọn một workflow cụ thể, viết Problem Statement, đánh giá AI readiness, và quyết định nên để rule/workflow, LLM feature, hay agent

Deliverable: 1-page artifact pack:

Problem Statement + RACI-lite

+ AI Readiness + Go/No-Go

+ next experiment

Thời gian: 90 phút

Những ý chính cần nhớ trước khi sang bài tiếp theo

- 1 Đúng architecture là quyết định quan trọng hơn đúng model. Bắt đầu từ rule / work-flow trước khi nhảy lên agent.
- 2 Lifecycle của AI product là chuỗi gate: metric, baseline, eval, risk control, rollback.
- 3 Problem Statement tốt sẽ suy ra được eval plan, system boundary, và ownership.
- 4 Feasibility tốt nghĩa là khả thi về kỹ thuật, vận hành, và business – không chỉ là model chạy được.

Từ Chatbot Đến Agentic Agent - Design Pattern ReAct

“Hôm nay bạn đã học cách quyết định khi nào cần agent. Ngày mai: nếu đã cần agent, nó suy nghĩ, gọi tool, và loop như thế nào?”

- Đọc artifact của nhóm khác và chỉ ra 1 lỗ hổng về metric hoặc boundary
- Thử đặt 1 use case trong công việc của bạn vào 3 mức: rule, LLM feature, agent

- 1 McKinsey & Company. *The State of AI in 2025*. 2025.
- 2 Google PAIR. *People + AI Guidebook*. pair.withgoogle.com/guidebook-v2/
- 3 Google for Developers. *Rules of Machine Learning*.
developers.google.com/machine-learning/guides/rules-of-ml
- 4 Sculley et al. *Hidden Technical Debt in Machine Learning Systems*. NIPS 2015.
- 5 OpenAI. *Agent evals and Safety in building agents*. developers.openai.com
- 6 Anthropic. *Building Effective AI Agents*. resources.anthropic.com

Hỏi & Đáp

*Bài toán nào trong team của bạn đang được gán nhãn “agent”
nhưng thực ra mới chỉ cần workflow hoặc LLM feature?*



Cảm ơn!

Email: lecturer@vinuni.edu.vn

Slides & tài liệu: github.com/aicb-vinuni

Lab template: bit.ly/aicb-day02-lab